Istio Traffic Management Lab Report

Objective

This lab exercise provides hands-on experience with Istio's traffic management capabilities, including traffic shifting, request routing, fault injection, and circuit breaking. These techniques demonstrate how to manage deployments and migrations of service versions in a microservices architecture.

Prerequisites

- Kubernetes cluster (Minikube or cloud-based)
- Istio installed on the cluster (istioctl install --set profile=demo -y)
- kubectl configured to communicate with the cluster
- Bookinfo sample application deployed in the default namespace

Part 1: Traffic Shifting

Goal

Gradually shift traffic from version v1 to v2 of a service using weighted routing.

Steps and Observations

1.1 Initial Setup – Deploy with All Traffic to v1

First, I deployed the Bookinfo application and created DestinationRules to define service subsets.

I then created a VirtualService to route 100% of traffic to reviews:v1

After applying this configuration, refreshing the /productpage multiple times showed no star ratings, confirming all traffic went to v1.

1.2 Gradual Traffic Shift

I modified the VirtualService to shift 90% traffic to v2

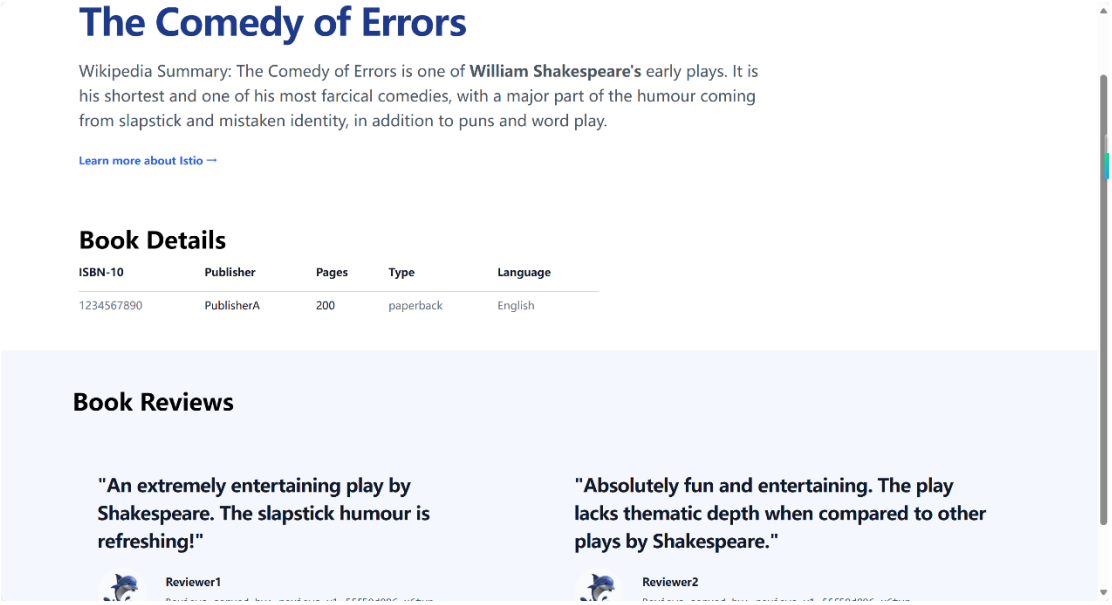
After applying the change, approximately 90% of page refreshes displayed star ratings, demonstrating successful traffic shifting.

```
u6237@Yezo:~$ kubectl get crd gateways.gateway.networking.k8s.io &> /dev/null || \
  kubectl apply --server-side -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/experimental-install.yaml
u6237@Yezo:~$ kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
error: the path "samples/bookinfo/networking/virtual-service-all-v1.yaml" does not exist
u6237@Yezo:~$ cd istio-1.29.2
u6237@Yezo:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage created
virtualservice.networking.istio.io/reviews created
virtualservice.networking.istio.io/ratings created
virtualservice.networking.istio.io/details created
u6237@Yezo:~/istio-1.29.2$ export GATEWAY_URL=$(minikube ip):$(kubectl get svc istio-ingressgateway -n istio-system -o jsonpath='{.spec.ports[?(@.na
me=="http2")].nodePort}')
u6237@Yezo:~/istio-1.29.2$ echo $GATEWAY_URL
192.168.49.2:30093
```

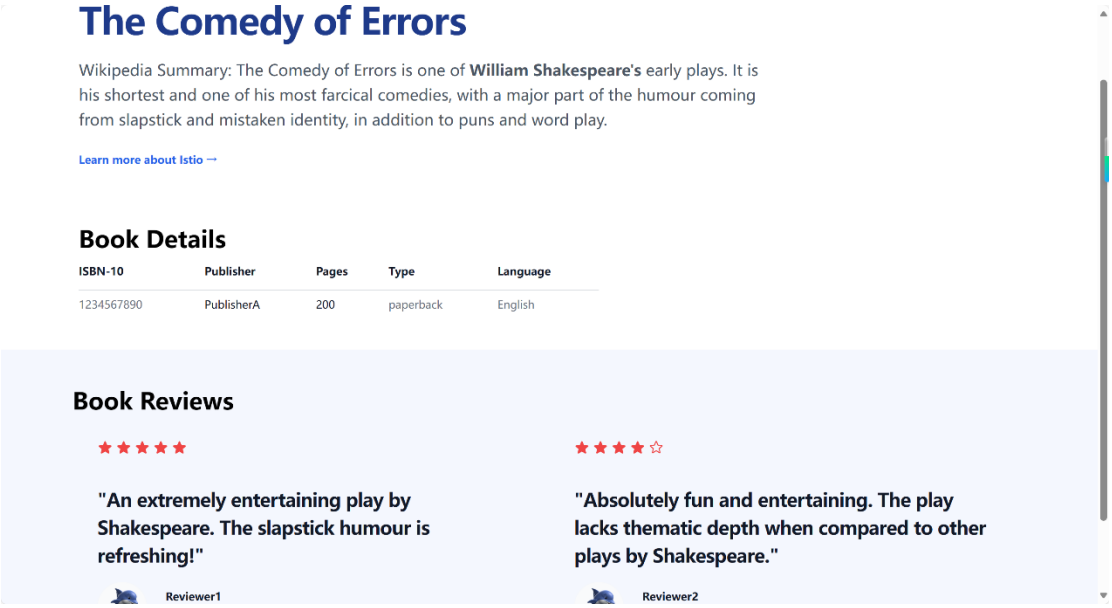
```
u6237@Yezo:~/istio-1.29.2$ kubectl port-forward -n istio-system svc/istio-ingressgateway 8080:80
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Handling connection for 8080
Handling connection for 8080
E0514 20:22:04.285843   14540 portforward.go:522] "Unhandled Error" err=<
        an error occurred forwarding 8080 -> 8080: error forwarding port 8080 to pod b613b80f73f6dc0024c700ff12a4f55964a4fad892b017dc9d436a6a31a8e2b
2, uid : exit status 1: 2026/05/14 12:22:04 socat[11868] E connect(5, AF=2 127.0.0.1:8080, 16): Connection refused
 >
error: lost connection to pod
```

There is an error for connection， we use another way to solve it:

```
u6237@Yezo:~/istio-1.29.2$ PRODUCT_POD=$(kubectl get pods -l app=productpage -o jsonpath='{.items[0].metadata.name}')
u6237@Yezo:~/istio-1.29.2$ kubectl port-forward $PRODUCT_POD 8888:9080
Forwarding from 127.0.0.1:8888 -> 9080
Forwarding from [::1]:8888 -> 9080
Handling connection for 8888
Handling connection for 8888
```




```
^Cu6237@Yezo:~/istio-1.29.2kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-50-v3.yamlml
virtualservice.networking.istio.io/reviews configured
u6237@Yezo:~/istio-1.29.2$ kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":{"h
osts":["reviews"],"http":[{"route":[{"destination":{"host":"reviews","subset":"v1"},"weight":50},{"destination":{"host":"reviews","subset":"v3"},"we
ight":50}]}]}}
  creationTimestamp: "2026-05-14T12:07:23Z"
  generation: 2
  name: reviews
  namespace: default
  resourceVersion: "56112"
  uid: bd378658-f693-4b95-8fb1-0994c136a7bb
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
      weight: 50
    - destination:
        host: reviews
        subset: v3
      weight: 50
```



Part 2: Request Routing

Goal

Route traffic based on HTTP request headers to enable canary deployments and A/B testing.

Steps and Observations

I configured Istio to route traffic from user "jason" to reviews:v2 while sending all other users to reviews:v1.

Verification

- When I logged in as "jason", the product page displayed star ratings (reviews:v2)
- When logged in as any other user, no star ratings appeared (reviews:v1)

This confirms that Istio successfully routes based on HTTP header values, enabling use cases like A/B testing and user segmentation.

```
^Cu6237@Yezo:~/istio-1.29.2kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage unchanged
virtualservice.networking.istio.io/reviews configured
virtualservice.networking.istio.io/ratings unchanged
virtualservice.networking.istio.io/details unchanged
u6237@Yezo:~/istio-1.29.2$ kubectl get virtualservices -o yaml
apiVersion: v1
items:
- apiVersion: networking.istio.io/v1
  kind: VirtualService
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"bookinfo","namespace":"default"},"spec":
{"gateways":["bookinfo-gateway"],"hosts":["*"],"http":[{"match":[{"uri":{"exact":"/productpage"}},{"uri":{"prefix":"/static"}},{"uri":{"exact":"/log
in"}},{"uri":{"exact":"/logout"}},{"uri":{"prefix":"/api/v1/products"}}],"route":[{"destination":{"host":"productpage","port":{"number":9080}}}]}]}}
    creationTimestamp: "2026-05-14T13:01:22Z"
    generation: 1
    name: bookinfo
    namespace: default
    resourceVersion: "57248"
    uid: 0cd75721-d4e3-4603-9214-fe998f90b422
  spec:
    gateways:
    - bookinfo-gateway
    hosts:
    - '*'
    http:
    - match:
      - uri:
          exact: /productpage
      - uri:
          prefix: /static
      - uri:
          exact: /login
      - uri:
          exact: /logout
      - uri:
          prefix: /api/v1/products
```

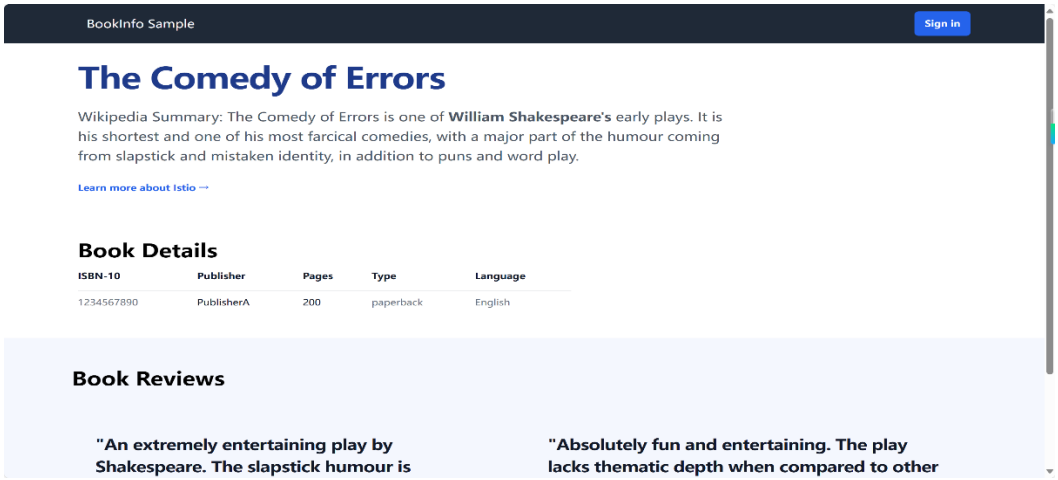
```
          prefix: /api/v1/products
      route:
      - destination:
          host: productpage
          port:
            number: 9080
- apiVersion: networking.istio.io/v1
  kind: VirtualService
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"details","namespace":"default"},"spec":{
"hosts":["details"],"http":[{"route":[{"destination":{"host":"details","subset":"v1"}}]}]}}
    creationTimestamp: "2026-05-14T13:03:30Z"
    generation: 1
    name: details
    namespace: default
    resourceVersion: "57445"
    uid: 24241e2f-a460-4032-94d4-f32e2bec4997
  spec:
    hosts:
    - details
    http:
    - route:
      - destination:
          host: details
          subset: v1
- apiVersion: networking.istio.io/v1
  kind: VirtualService
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"productpage","namespace":"default"},"spe
c":{"hosts":["productpage"],"http":[{"route":[{"destination":{"host":"productpage","subset":"v1"}}]}]}}
    creationTimestamp: "2026-05-14T13:03:30Z"
    generation: 1
    name: productpage
    namespace: default
    resourceVersion: "57442"
    uid: 91ec5e76-db97-4eee-8b16-8413f8dbfabe
  spec:
    hosts:
    - productpage
```

```
    - productpage
    http:
    - route:
      - destination:
          host: productpage
          subset: v1
- apiVersion: networking.istio.io/v1
  kind: VirtualService
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"ratings","namespace":"default"},"spec":{
"hosts":["ratings"],"http":[{"route":[{"destination":{"host":"ratings","subset":"v1"}}]}]}}
    creationTimestamp: "2026-05-14T13:03:30Z"
    generation: 1
    name: ratings
    namespace: default
    resourceVersion: "57444"
    uid: 4b7244e2-c5fc-4b6e-a1c3-220c57ff2271
  spec:
    hosts:
    - ratings
    http:
    - route:
      - destination:
          host: ratings
          subset: v1
- apiVersion: networking.istio.io/v1
  kind: VirtualService
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":{
"hosts":["reviews"],"http":[{"route":[{"destination":{"host":"reviews","subset":"v1"}}]}]}}
    creationTimestamp: "2026-05-14T13:03:30Z"
    generation: 4
    name: reviews
    namespace: default
    resourceVersion: "57912"
    uid: 870d7386-768b-4d20-85b8-58e4c7ba886e
  spec:
    hosts:
    - reviews
```

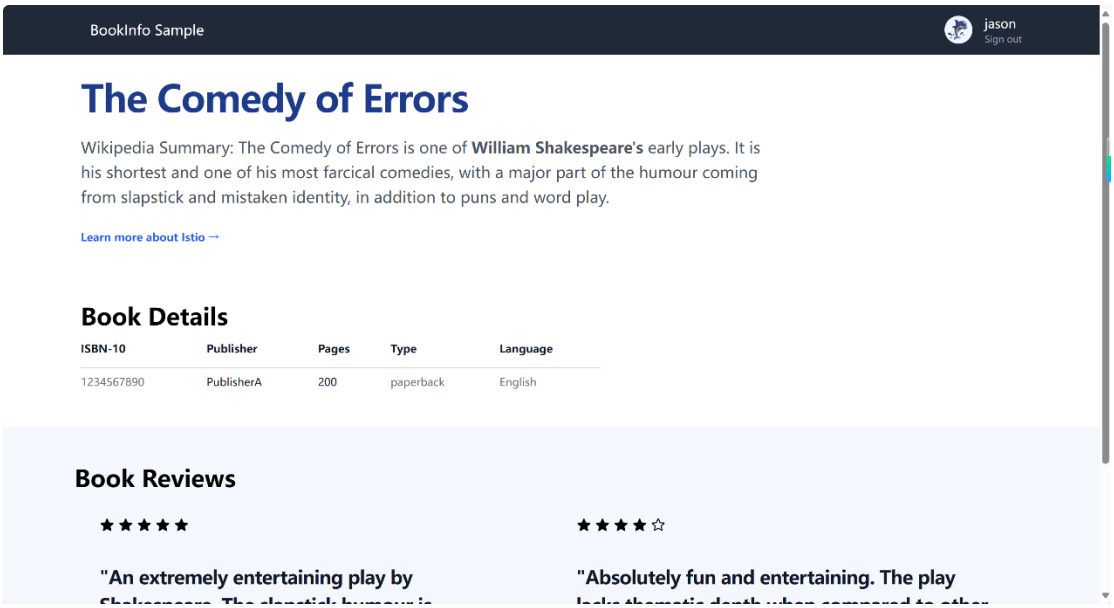
```
    http:
    - route:
      - destination:
          host: reviews
          subset: v1
kind: List
metadata:
  resourceVersion: ""
u6237@Yezo:~/istio-1.29.2$ kubectl get destinationrules -o yaml
apiVersion: v1
items:
- apiVersion: networking.istio.io/v1
  kind: DestinationRule
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"DestinationRule","metadata":{"annotations":{},"name":"details","namespace":"default"},"spec":
{"host":"details","subsets":[{"labels":{"version":"v1"},"name":"v1"},{"labels":{"version":"v2"},"name":"v2"}]}}
    creationTimestamp: "2026-05-14T13:01:29Z"
    generation: 1
    name: details
    namespace: default
    resourceVersion: "57263"
    uid: a858a189-f585-483a-b920-a742da26b5ea
  spec:
    host: details
    subsets:
    - labels:
        version: v1
      name: v1
    - labels:
        version: v2
      name: v2
- apiVersion: networking.istio.io/v1
  kind: DestinationRule
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"DestinationRule","metadata":{"annotations":{},"name":"productpage","namespace":"default"},"sp
ec":{"host":"productpage","subsets":[{"labels":{"version":"v1"},"name":"v1"}]}}
    creationTimestamp: "2026-05-14T13:01:29Z"
    generation: 1
```

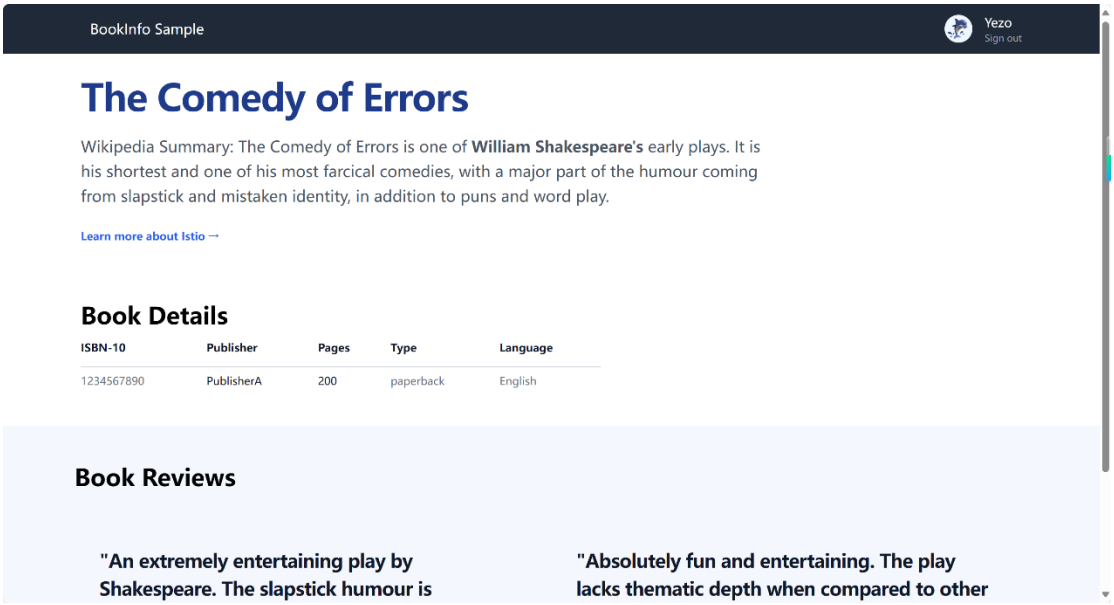
```
    name: productpage
    namespace: default
    resourceVersion: "57260"
    uid: 862bd7e3-9e81-4443-8cce-2bf3bb6e3b6e
  spec:
    host: productpage
    subsets:
    - labels:
        version: v1
      name: v1
- apiVersion: networking.istio.io/v1
  kind: DestinationRule
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"DestinationRule","metadata":{"annotations":{},"name":"ratings","namespace":"default"},"spec":
{"host":"ratings","subsets":[{"labels":{"version":"v1"},"name":"v1"},{"labels":{"version":"v2"},"name":"v2"},{"labels":{"version":"v2-mysql"},"name"
:"v2-mysql"},{"labels":{"version":"v2-mysql-vm"},"name":"v2-mysql-vm"}]}}
    creationTimestamp: "2026-05-14T13:01:29Z"
    generation: 1
    name: ratings
    namespace: default
    resourceVersion: "57262"
    uid: 35d96d79-5523-4d56-9b82-154f63eba22b
  spec:
    host: ratings
    subsets:
    - labels:
        version: v1
      name: v1
    - labels:
        version: v2
      name: v2
    - labels:
        version: v2-mysql
      name: v2-mysql
    - labels:
        version: v2-mysql-vm
      name: v2-mysql-vm
- apiVersion: networking.istio.io/v1
  kind: DestinationRule
  metadata:
```

```
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"DestinationRule","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":
{"host":"reviews","subsets":[{"labels":{"version":"v1"},"name":"v1"},{"labels":{"version":"v2"},"name":"v2"},{"labels":{"version":"v3"},"name":"v3"}
]}}
    creationTimestamp: "2026-05-14T13:01:29Z"
    generation: 1
    name: reviews
    namespace: default
    resourceVersion: "57261"
    uid: 7a8df720-17a9-4aa0-b253-0077ee3223a4
  spec:
    host: reviews
    subsets:
    - labels:
        version: v1
      name: v1
    - labels:
        version: v2
      name: v2
    - labels:
        version: v3
      name: v3
kind: List
metadata:
  resourceVersion: ""
```




```
^Cu6237@Yezo:~/istio-1.29.2kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
virtualservice.networking.istio.io/reviews configured
u6237@Yezo:~/istio-1.29.2$ kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":{"hosts":["reviews"]
,"http":[{"match":[{"headers":{"end-user":{"exact":"jason"}}}],"route":[{"destination":{"host":"reviews","subset":"v2"}}]},{"route":[{"destination":{"host":"reviews"
,"subset":"v1"}}]}]}}
  creationTimestamp: "2026-05-14T13:03:30Z"
  generation: 5
  name: reviews
  namespace: default
  resourceVersion: "58330"
  uid: 870d7386-768b-4d20-85b8-58e4c7ba886e
spec:
  hosts:
  - reviews
  http:
  - match:
    - headers:
        end-user:
          exact: jason
    route:
    - destination:
        host: reviews
        subset: v2
  - route:
    - destination:
        host: reviews
        subset: v1
```





Part 3: Fault Injection

Goal

Test application resilience by injecting delays and abort failures.

Steps and Observations

3.1 Injecting a 7-second Delay

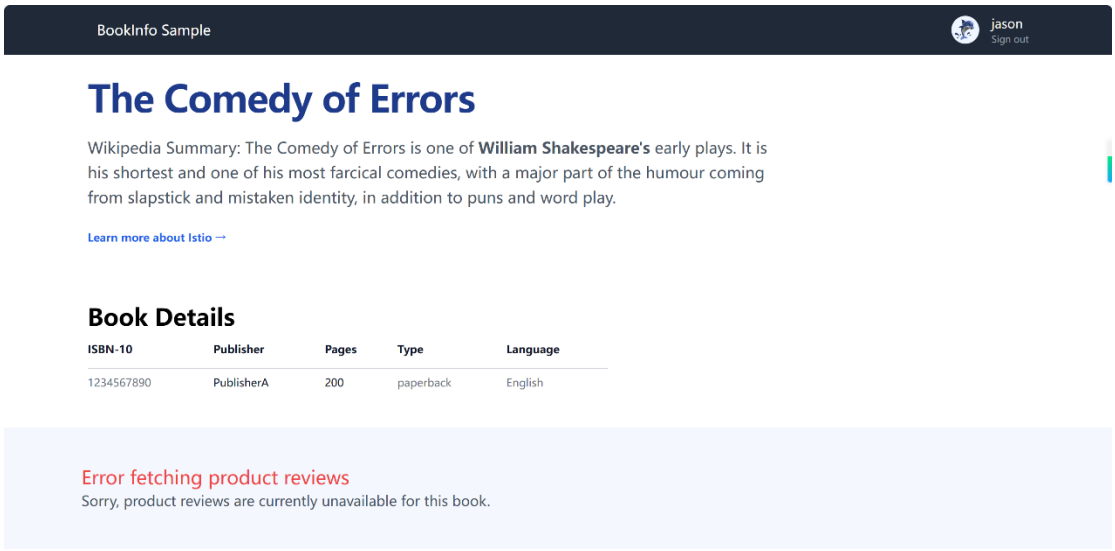
I configured a fault injection rule to add a 7-second delay to 100% of requests from user "jason" to reviews:v2

After applying this configuration, logging in as "jason" caused the product page to take approximately 7 seconds to load. The browser developer tools clearly showed the delay in the network tab.

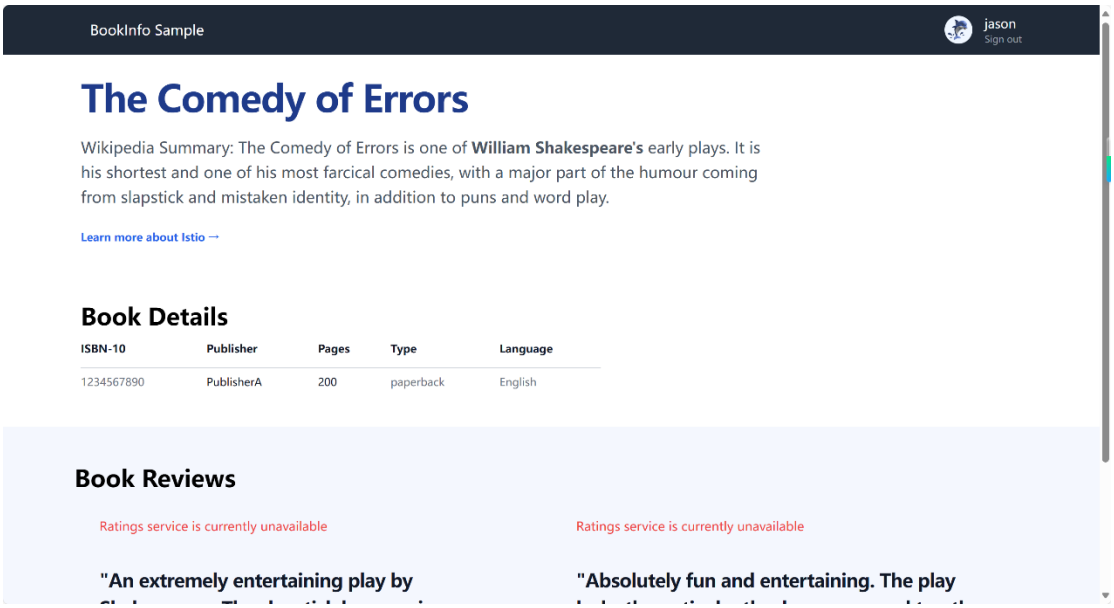
3.2 Injecting an HTTP Abort

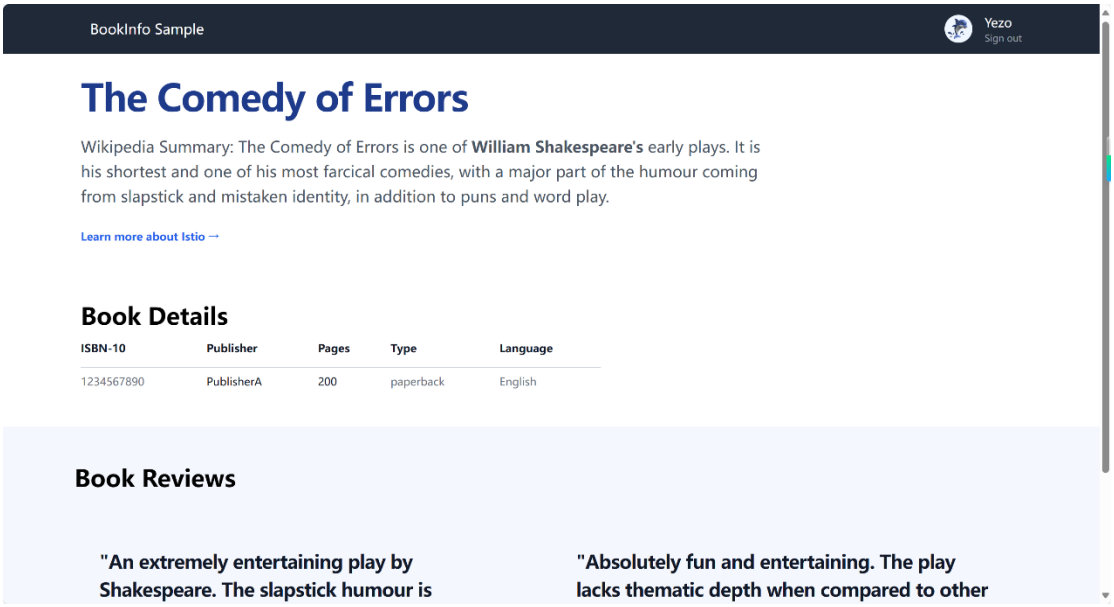
I then modified the fault injection to return an HTTP 500 error instead of a delay Logging in as "jason" now produced an empty page with an error in the browser console. The reviews service failed to load, demonstrating how fault injection helps test error handling.

```
u6237@Yezo:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-delay.yaml
virtualservice.networking.istio.io/ratings configured
u6237@Yezo:~/istio-1.29.2$ kubectl get virtualservice ratings -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"ratings","namespace":"default"},"spec":{"hosts":["ratings"]
,"http":[{"fault":{"delay":{"fixedDelay":"7s","percentage":{"value":100}}},"match":[{"headers":{"end-user":{"exact":"jason"}}}],"route":[{"destination":{"host":"rati
ngs","subset":"v1"}}]},{"route":[{"destination":{"host":"ratings","subset":"v1"}}]}]}}
  creationTimestamp: "2026-05-14T13:03:30Z"
  generation: 2
  name: ratings
  namespace: default
  resourceVersion: "59015"
  uid: 4b7244e2-c5fc-4b6e-a1c3-220c57ff2271
spec:
  hosts:
  - ratings
  http:
  - fault:
      delay:
        fixedDelay: 7s
        percentage:
          value: 100
    match:
    - headers:
        end-user:
          exact: jason
    route:
    - destination:
        host: ratings
        subset: v1
  - route:
    - destination:
        host: ratings
        subset: v1
```




```
^Cu6237@Yezo:~/istio-1.29.2kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-abort.yaml
virtualservice.networking.istio.io/ratings configured
u6237@Yezo:~/istio-1.29.2$ kubectl get virtualservice ratings -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"ratings","namespace":"default"},"spec":{"hosts":["ratings"]
,"http":[{"fault":{"abort":{"httpStatus":500,"percentage":{"value":100}}},"match":[{"headers":{"end-user":{"exact":"jason"}}}],"route":[{"destination":{"host":"ratin
gs","subset":"v1"}}]},{"route":[{"destination":{"host":"ratings","subset":"v1"}}]}]}}
  creationTimestamp: "2026-05-14T13:03:30Z"
  generation: 3
  name: ratings
  namespace: default
  resourceVersion: "59666"
  uid: 4b7244e2-c5fc-4b6e-a1c3-220c57ff2271
spec:
  hosts:
  - ratings
  http:
  - fault:
      abort:
        httpStatus: 500
        percentage:
          value: 100
    match:
    - headers:
        end-user:
          exact: jason
    route:
    - destination:
        host: ratings
        subset: v1
  - route:
    - destination:
        host: ratings
        subset: v1
```





Part 4: Circuit Breaking

Goal

Configure circuit breaking to prevent cascading failures by limiting connections to a service.

Steps and Observations

I deployed the httpbin sample application and configured a DestinationRule with circuit breaking settings

To test the circuit breaker, I used the Fortio load testing tool

Results

- Only 1 connection was active; the second connection was rejected
- Approximately 50% of requests succeeded, 50% failed (connection errors)

- The outlier detection ejected unhealthy hosts after consecutive 5xx errors

This demonstrates that circuit breaking effectively protects services from overload by limiting concurrent connections and rejecting unhealthy endpoints.

```
Handling connection for 8888
^Cu6237@Yezo:~/istio-1.29.2kubectl apply -f samples/httpbin/httpbin.yamlml
serviceaccount/httpbin created
service/httpbin created
deployment.apps/httpbin created
```

```
u6237@Yezo:~/istio-1.29.2$ kubectl apply -f - <<EOF
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: httpbin
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 1
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
    outlierDetection:
      consecutive5xxErrors: 1
      interval: 1s
      baseEjectionTime: 3m
      maxEjectionPercent: 100
EOF
destinationrule.networking.istio.io/httpbin created
```

```
u6237@Yezo:~/istio-1.29.2$ kubectl get destinationrule httpbin -o yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"DestinationRule","metadata":{"annotations":{},"name":"httpbin","namespace":"default"},"spec":{"host":"httpbin","
trafficPolicy":{"connectionPool":{"http":{"http1MaxPendingRequests":1,"maxRequestsPerConnection":1},"tcp":{"maxConnections":1}},"outlierDetection":{"baseEjectionTime
":"3m","consecutive5xxErrors":1,"interval":"1s","maxEjectionPercent":100}}}}
  creationTimestamp: "2026-05-14T13:33:52Z"
  generation: 1
  name: httpbin
  namespace: default
  resourceVersion: "60202"
  uid: 23bd8867-4723-487b-8511-76e7bc94891c
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
      tcp:
        maxConnections: 1
    outlierDetection:
      baseEjectionTime: 3m
      consecutive5xxErrors: 1
      interval: 1s
      maxEjectionPercent: 100
```

```
u6237@Yezo:~/istio-1.29.2$ kubectl apply -f samples/httpbin/sample-client/fortio-deploy.yaml
service/fortio created
deployment.apps/fortio-deploy created
```

```
u6237@Yezo:~/istio-1.29.2$ kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio curl -quiet http://httpbin:8000/get
HTTP/1.1 200 OK
server: envoy
date: Tue, 25 Feb 2020 20:25:52 GMT
content-type: application/json
content-length: 586
access-control-allow-origin: *
access-control-allow-credentials: true
x-envoy-upstream-service-time: 36

{
  "args": {},
  "headers": {
    "Content-Length": "0",
    "Host": "httpbin:8000",
    "User-Agent": "fortio.org/fortio-1.3.1",
    "X-B3-Parentspanid": "8fc453fb1dec2c22",
    "X-B3-Sampled": "1",
    "X-B3-Spanid": "071d7f06bc94943c",
    "X-B3-Traceid": "86a929a0e76cda378fc453fb1dec2c22",
    "X-Forwarded-Client-Cert": "By=spiffe://cluster.local/ns/default/sa/httpbin;Hash=68bbaedefe01ef4cb99e17358ff63e92d04a4ce831a35ab9a31d3c8e06adb03
8;Subject=\"\";URI=spiffe://cluster.local/ns/default/sa/default"
  },
  "origin": "127.0.0.1",
  "url": "http://httpbin:8000/get"
}
```

```
u6237@Yezo:~/istio-1.29.2$ kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio load -c 2 -qps 0 -n 20 -loglevel Warning http://httpbin:8000/get
20:33:46 I logger.go:97> Log level is now 3 Warning (was 2 Info)
Fortio 1.3.1 running at 0 queries per second, 6->6 procs, for 20 calls: http://httpbin:8000/get
Starting at max qps with 2 thread(s) [gomax 6] for exactly 20 calls (10 per thread + 0)
20:33:46 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:33:47 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:33:47 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
Ended after 59.8524ms : 20 calls. qps=334.16
Aggregated Function Time : count 20 avg 0.0056869 +/- 0.003869 min 0.000499 max 0.0144329 sum 0.113738
# range, mid point, percentile, count
>= 0.000499 <= 0.001 , 0.0007495 , 10.00, 2
> 0.001 <= 0.002 , 0.0015 , 15.00, 1
> 0.003 <= 0.004 , 0.0035 , 45.00, 6
> 0.004 <= 0.005 , 0.0045 , 55.00, 2
> 0.005 <= 0.006 , 0.0055 , 60.00, 1
> 0.006 <= 0.007 , 0.0065 , 70.00, 2
> 0.007 <= 0.008 , 0.0075 , 80.00, 2
> 0.008 <= 0.009 , 0.0085 , 85.00, 1
> 0.011 <= 0.012 , 0.0115 , 90.00, 1
> 0.012 <= 0.014 , 0.013 , 95.00, 1
> 0.014 <= 0.0144329 , 0.0142165 , 100.00, 1
# target 50% 0.0045
# target 75% 0.0075
# target 90% 0.012
# target 99% 0.0143463
# target 99.9% 0.0144242
Sockets used: 4 (for perfect keepalive, would be 2)
Code 200 : 17 (85.0 %)
Code 503 : 3 (15.0 %)
Response Header Sizes : count 20 avg 195.65 +/- 82.19 min 0 max 231 sum 3913
Response Body/Total Sizes : count 20 avg 729.9 +/- 205.4 min 241 max 817 sum 14598
All done 20 calls (plus 0 warmup) 5.687 ms avg, 334.2 qps
```

```
u6237@Yezo:~/istio-1.29.2$ kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio load -c 3 -qps 0 -n 30 -loglevel Warning http://httpbin:8000/get
20:32:30 I logger.go:97> Log level is now 3 Warning (was 2 Info)
Fortio 1.3.1 running at 0 queries per second, 6->6 procs, for 30 calls: http://httpbin:8000/get
Starting at max qps with 3 thread(s) [gomax 6] for exactly 30 calls (10 per thread + 0)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
20:32:30 W http_client.go:679> Parsed non ok code 503 (HTTP/1.1 503)
Ended after 51.9946ms : 30 calls. qps=576.98
Aggregated Function Time : count 30 avg 0.0040001633 +/- 0.003447 min 0.0004298 max 0.015943 sum 0.1200049
# range, mid point, percentile, count
>= 0.0004298 <= 0.001 , 0.0007149 , 16.67, 5
> 0.001 <= 0.002 , 0.0015 , 36.67, 6
> 0.002 <= 0.003 , 0.0025 , 50.00, 4
> 0.003 <= 0.004 , 0.0035 , 60.00, 3
> 0.004 <= 0.005 , 0.0045 , 66.67, 2
> 0.005 <= 0.006 , 0.0055 , 76.67, 3
> 0.006 <= 0.007 , 0.0065 , 83.33, 2
> 0.007 <= 0.008 , 0.0075 , 86.67, 1
> 0.008 <= 0.009 , 0.0085 , 90.00, 1
> 0.009 <= 0.01 , 0.0095 , 96.67, 2
> 0.014 <= 0.015943 , 0.0149715 , 100.00, 1
# target 50% 0.003
# target 75% 0.00583333
# target 90% 0.009
# target 99% 0.0153601
# target 99.9% 0.0158847
Sockets used: 20 (for perfect keepalive, would be 3)
```

```
Sockets used: 20 (for perfect keepalive, would be 3)
All done 30 calls (plus 0 warmup) 4.000 ms avg, 577.0 qps7.1 min 241 max 817 sum 13556
```

```
u6237@Yezo:~/istio-1.29.2$ kubectl exec "$FORTIO_POD" -c istio-proxy -- pilot-agent request GET stats | grep httpbin | grep pending
cluster.outbound|8000||httpbin.default.svc.cluster.local;.circuit_breakers.default.rq_pending_open: 0
cluster.outbound|8000||httpbin.default.svc.cluster.local;.circuit_breakers.high.rq_pending_open: 0
cluster.outbound|8000||httpbin.default.svc.cluster.local;.upstream_rq_pending_active: 0
cluster.outbound|8000||httpbin.default.svc.cluster.local;.upstream_rq_pending_failure_eject: 0
cluster.outbound|8000||httpbin.default.svc.cluster.local;.upstream_rq_pending_overflow: 21
cluster.outbound|8000||httpbin.default.svc.cluster.local;.upstream_rq_pending_total: 29
```

Summary of Key Learnings

Feature	**Istio Resource**	**Key Use Case**
Traffic Shifting	VirtualService (weights)	Canary deployments, blue-green rollouts
Request Routing	VirtualService (matches)	A/B testing, user segmentation
Fault Injection	VirtualService (fault)	Resilience testing, chaos engineering
Circuit Breaking	DestinationRule	Overload protection, failure isolation

Conclusion

Istio provides powerful traffic management capabilities that enable safe deployment strategies and robust microservice architectures. Through this lab, I successfully:

1. **Shifted traffic gradually** between service versions using weighted routing
2. **Routed requests dynamically** based on HTTP headers for A/B testing
3. **Injected faults** to test application resilience
4. **Configured circuit breakers** to prevent cascading failures

These techniques are essential for production-ready microservices, allowing teams to deploy with confidence and maintain system stability under failure conditions.